

# **Waveframe System Overview**

Runtime Execution-Boundary Enforcement Architecture

Version 0.1.0

DOI: 10.5281/zenodo.20310076

Shawn C. Wright

Waveframe Labs

May 2026

# Waveframe System Overview

## 1. The Problem

Modern software systems increasingly combine probabilistic proposal generation with growing authority to perform consequential actions. AI systems, automation frameworks, workflow engines, and human-in-the-loop processes can now generate candidate mutations at high speed across operational environments.

Traditional oversight models assume that humans remain the primary execution chokepoints. Policies are written as documents, approvals live in review processes, and compliance is often reconstructed after the fact through logs, tickets, and audit trails.

That assumption is weakening.

The central problem is not that probabilistic systems generate proposals. Proposal generation is useful and should remain flexible. The problem is that consequential execution needs deterministic authority, stable evaluation boundaries, and reproducible evidence. A system that can suggest a database change, financial transfer, deployment, access escalation, or policy mutation must be evaluated at the point where suggestion becomes action.

Execution control therefore has to move closer to the mutation boundary.

## 2. The Architectural Shift

Historically, institutional control has been document-centric. Organizations define policies, publish procedures, train operators, and later audit whether actions complied with those expectations.

That model is too slow and ambiguous for systems where proposed actions may be generated continuously by humans, agents, automations, or orchestration layers.

The architectural shift is that governance becomes infrastructure.

Runtime authority must become:

- executable
- machine-evaluable
- runtime-resolvable
- execution-adjacent
- replayable after the fact

This does not require organizations to discard existing policy documents or governance processes. Existing governance artifacts can serve as source authority for deterministic compilation, structured review, and runtime admissibility evaluation.

This does not mean replacing human judgment. It means converting approved policy into deterministic runtime authority that systems can evaluate before consequential execution occurs.

### 3. The Separation Principle

The core abstraction is:

proposal generation != execution authority

The system does not attempt to prevent proposal generation. People, AI systems, workflow tools, and automation engines may all generate candidate actions.

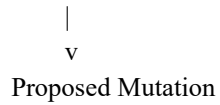
The system governs whether those proposals are admissible for consequential execution.

That separation matters because it applies broadly. The same enforcement boundary can sit between:

- an AI agent and a mutation
- a human operator and a privileged action
- a workflow engine and a deployment
- an automation script and a production system
- an orchestration layer and a regulated transaction

The proposal layer remains creative and flexible. The execution boundary remains deterministic, explicit, and auditable.

Probabilistic Proposal Generation  
(AI / Human / Workflow / Agent)



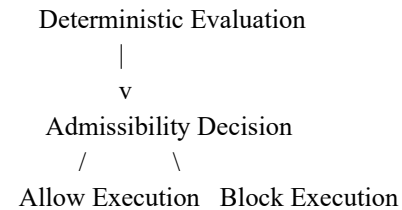
---

---

EXECUTION BOUNDARY

---

---



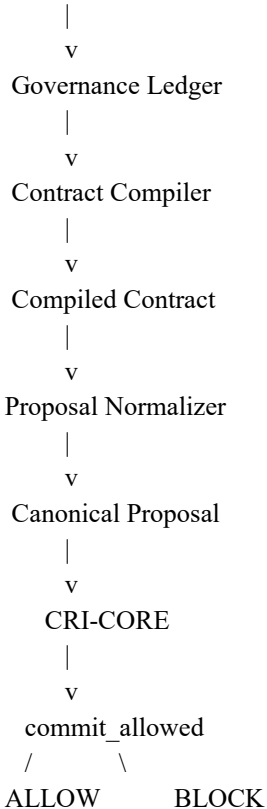
#### 4. Waveframe Architecture

Waveframe is organized around responsibility boundaries rather than a single monolithic policy engine.

| Component           | Role                                                                                                             |
|---------------------|------------------------------------------------------------------------------------------------------------------|
| Governance Ledger   | Tracks governance source, review state, publication lineage, snapshots, rollback, and replay evidence.           |
| Compiler            | Converts approved governance into deterministic authority contracts.                                             |
| Proposal Normalizer | Produces canonical machine-evaluable proposal objects.                                                           |
| CRI-CORE            | Evaluates structured proposals against authority constraints using deterministic sequencing.                     |
| Guard               | Owns runtime interception, local admissibility gating, execution posture, and fail-closed behavior.              |
| Cloud               | Provides operational distribution, durability, and replay support without becoming the execution decision-maker. |

The important boundary is that runtime admissibility remains local and deterministic. Cloud infrastructure may distribute authority artifacts and preserve operational evidence, but admissibility evaluation remains locally owned by the governed runtime.

Policy / Governance Source



## 5. Runtime Enforcement Model

Runtime enforcement begins at the execution boundary.

Before a consequential action is allowed to proceed, the system resolves the relevant authority, validates its identity and lineage, normalizes the proposed mutation into a stable structure, and evaluates the proposal through a deterministic sequence of checks.

In simplified form:

proposal

- > authority resolution
- > contract verification
- > admissibility evaluation
- > allow/block decision
- > governed evidence

The evaluation pipeline answers one explicit question:

Is this proposal admissible for execution under the bound authority?

The result is expressed as an explicit allow/block signal, including a `commit_allowed` decision where applicable.

Important properties:

- evaluation order is deterministic
- authority is explicit
- proposal and authority are bound together
- missing or invalid authority fails closed
- runtime decisions produce evidence
- replay can reproduce the decision path

Deterministic sequencing ensures identical authority, proposal, and runtime conditions produce reproducible admissibility outcomes.

The model is intentionally narrow. It does not ask whether a proposal is useful, creative, or strategically wise. It asks whether the proposal is admissible under the governing authority at the execution boundary.

## 6. Proposal Normalization

Proposal normalization is not governance interpretation.

Its purpose is to create stable executable structure.

Runtime authority requires objects that can be evaluated consistently. Natural language, ad hoc payloads, and tool-specific request shapes are too unstable to serve as the enforcement substrate.

The normalizer binds the key elements of a proposed action into canonical form:

- actor binding
- mutation binding
- contract binding
- artifact references
- deterministic representation
- artifact hashing

This creates a machine-evaluable object that can be inspected, hashed, replayed, and evaluated against authority constraints.

The normalizer does not decide whether the proposal should execute. It makes the proposal structurally legible to the enforcement layer.

## 7. Governed Execution

Governed execution describes what happens operationally when a system attempts to perform a consequential action.

The execution path is intercepted or wrapped by a runtime guard. The guard resolves the bound authority, evaluates admissibility, and then either allows the action to proceed or blocks execution before the mutation occurs.

The guard owns the execution posture:

- intercept execution
- bind actor and action context
- evaluate admissibility
- allow or block execution
- emit governed evidence
- fail closed when authority or integrity is invalid

This keeps execution ownership clear. Applications and automation systems may request action. The governed runtime determines whether the action may cross the execution boundary.

## 8. Operational Lineage

A serious governance system must be able to explain how an execution decision came to exist.

Waveframe treats lineage as a first-class operational concern. Source policy, review state, compilation output, authority publication, runtime execution, and evidence receipts are linked into a reproducible chain.

This enables:

- provenance tracking
- replayability
- execution evidence
- deployment lineage
- governance state history
- rollback and snapshot workflows
- independent verification

The goal is not merely to log that something happened. The goal is to preserve enough structured evidence to determine whether the runtime authority, proposal, and execution decision were consistent at the time of action.

## 9. Architectural Invariants

The system is designed around hard invariants:

- Proposal generation is separate from execution authority.
- Runtime admissibility is evaluated deterministically.
- Authority must be explicit.
- Contract identity is immutable.
- Proposal objects are bound to authority.
- Invalid or missing authority fails closed.
- Revoked authority fails closed.
- Superseded authority does not silently replace the bound authority.
- Execution boundaries are owned by the governed runtime.
- Decisions produce explicit admissibility signals.
- Evidence must be replayable.
- Cloud infrastructure must not become a hidden execution dependency.

These invariants give the architecture hard edges. They are what distinguish runtime enforcement infrastructure from advisory policy tooling.



## 10. Open Problems

Several problems remain active areas of design and research:

- live governing state
- distributed authority resolution
- semantic portability across domains
- interoperability with external policy systems
- external state grounding
- multi-system execution boundaries
- partial trust between organizations
- long-running workflow authority
- human override semantics
- governance migration across versions

These are real problems. The architecture does not require pretending they are solved. It creates a deterministic enforcement and evidence foundation on which those problems can be addressed explicitly.

## 11. Conclusion

As probabilistic systems gain increasing ability to generate consequential mutations, governance must transition from advisory oversight into deterministic execution-boundary enforcement infrastructure.

Waveframe is built around that transition: proposals may remain flexible, but execution authority must be explicit, machine-evaluable, locally enforceable, and replayable.

## Appendix: Vocabulary

| Term                     | Definition                                                                                                            |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Proposal                 | A candidate action produced by a human, AI system, workflow, agent, or automation layer.                              |
| Mutation                 | A consequential change requested against a system, resource, process, or state.                                       |
| Admissibility            | Whether a proposal is allowed to cross the execution boundary under the bound authority.                              |
| Execution Boundary       | The point where a proposed action would become a consequential mutation.                                              |
| Governed Execution       | Runtime execution that is intercepted, evaluated, allowed or blocked, and recorded as evidence.                       |
| Authority                | The approved policy basis used to evaluate whether execution is permitted.                                            |
| Runtime Authority        | Machine-evaluable authority resolved by the runtime before execution.                                                 |
| Canonical Proposal       | A deterministic proposal representation with actor, mutation, artifact, and authority bindings.                       |
| Deterministic Sequencing | Evaluation ordering that ensures identical authority, proposal, and runtime conditions produce reproducible outcomes. |
| Operational Lineage      | The evidence chain connecting source policy, review, compiled authority, execution decision, and replay material.     |